РЕШЕН ИЗПИТ 2 — С ПЪЛНИ ОБЯСНЕНИЯ

Базови Програмни Езици — C (plain C)

Изпит 2: Болница — Медицинско досие | 100 точки

Теми: четене от бинарен файл с header-count | броене по две условия | realloc + return NULL | запис в текстов файл

Специфика: add_new_treatment() трябва да върне NULL при грешка — не exit()!

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ

КОД / ИЗРАЗ	КАКВО ОЗНАЧАВА
`ID (цяло число)`	int id;
`Дата ДД.ММ.ГГГГ`	char date[11]; — 10 + 1 = 11
`Пациент (до 30 символа)`	char name[31]; — 30 + 1 = 31
`Диагноза (до 50 символа)`	char diagnosis[51]; — 50 + 1 = 51
`файлът съдържа БРОЯ, последван от записите`	fread за count първо, после fread за масива
`Функцията връща NULL при грешка`	return NULL; — НЕ exit(1)!

СЕКЦИЯ 1 — ПЪЛЕН КОД (gcc -Wall -std=c99)

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int  id;
    char date[11];       /* ДД.ММ.ГГГГ = 10 + 1 */
    char name[31];       /* до 30 символа + 1 */
    char diagnosis[51];  /* до 50 символа + 1 */
} Treatment;

/* — Задача 1: четене от бинарен файл с header-count — */
Treatment* loadHistory(char *fn, int *n) {
    FILE *f = fopen(fn, "rb");
    if (!f) { printf("Error\n"); *n = 0; return NULL; }
    if (fread(n, sizeof(int), 1, f) != 1) {
        printf("Error\n"); fclose(f); *n = 0; return NULL;
    }
    Treatment *arr = malloc(*n * sizeof(Treatment));
    if (!arr) { printf("Error\n"); fclose(f); return NULL; }
    fread(arr, sizeof(Treatment), *n, f);
    fclose(f);
    return arr;
}

/* — Задача 2: count_treatment — */
int count_treatment(Treatment *arr, int n, char *name, char *diagnosis) {
    int cnt = 0;
    for (int i = 0; i < n; i++)
        if (strcmp(arr[i].name, name) == 0 &&
            strcmp(arr[i].diagnosis, diagnosis) == 0)
            cnt++;
    return cnt;
}

/* — Задача 3: add_new_treatment — */
Treatment* add_new_treatment(Treatment *arr, int *n) {
    Treatment *tmp = realloc(arr, (*n + 1) * sizeof(Treatment));
    if (!tmp) { printf("Error\n"); return NULL; }
    arr = tmp;
    (*n)++;
    Treatment *t = &arr[*n - 1];
    printf("ID: ");        scanf("%d",   &t->id);
    printf("Дата: ");      scanf("%10s",  t->date);
    printf("Пациент: ");   scanf("%30s",  t->name);
    printf("Диагноза: "); scanf("%50s",  t->diagnosis);
    return arr;
}

/* — Задача 4: write_text_file — */
int write_text_file(Treatment *arr, int n, char *diagnosis) {
    FILE *f = fopen("illness.txt", "w");
    if (!f) { printf("Error\n"); return 0; }
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        if (strcmp(arr[i].diagnosis, diagnosis) == 0) {
            fprintf(f, "Болничен престой на %s\n", arr[i].name);
            fprintf(f, "за лечение на %s:\n", arr[i].diagnosis);
```

```
            fprintf(f, "%sg.\n", arr[i].date);
            cnt++;
        }
    }
    fclose(f);
    return cnt;
}

int main() {
    int n = 0;
    Treatment *arr = loadHistory("history.bin", &n);
    if (!arr) return 1;
    printf("%d\n", count_treatment(arr, n, "Петър Иванов", "Дископатия"));
    arr = add_new_treatment(arr, &n);
    if (!arr) return 1;
    printf("%d\n", write_text_file(arr, n, "Дископатия"));
    free(arr);
    return 0;
}
```

СЕКЦИЯ 2 — ОБЯСНЕНИЯ РЕД ПО РЕД

loadHistory() — бинарен файл с header-count

Файлът history.bin съдържа ПЪРВО int с броя записи, а ПОСЛЕ самите структури. Нужни са два отделни fread.

```
/* Структура на history.bin:                          */
/* [ int n (4 байта) ][ Treatment 0 ][ Treatment 1 ] */
```

КОД	ОБЯСНЕНИЕ
`fread(n, sizeof(int), 1, f)`	Четем int брояча от началото на файла. Указателят се позиционира след 4-те байта.
`malloc(*n * sizeof(Treatment))`	Заделяме точно толкова памет колкото са записите. malloc (не realloc) — знаем размера предварително.
`fread(arr, sizeof(Treatment), *n, f)`	Четем ВСИЧКИ записи наведнъж. По-ефективно от цикъл с fread по 1.

add_new_treatment() — return NULL вместо exit()

Задачата изрично казва: при грешка функцията връща NULL. Не трябва exit(1)!

```
[NE]  if (!tmp) { exit(1); }            // НЕ за Изпит 2 — задачата иска NULL!
[OK]  if (!tmp) { printf("Error\n"); return NULL; }  // правилно
```

КОД	ОБЯСНЕНИЕ
`Treatment *tmp = realloc(arr, ...)`	Временна променлива tmp — ако realloc се провали, arr остава валиден.
`if (!tmp) { return NULL; }`	Нулев указател при грешка. Старият arr НЕ се губи.
`arr = tmp; (*n)++;`	Само СЛЕД успешен realloc обновяваме arr и брояча.

Грешки характерни за Изпит 2

```
[NE]  if (!tmp) { exit(1); }                 // задачата иска return NULL!
[OK]  if (!tmp) { printf("Error\n"); return NULL; }
```

```
[NE]  if (!f) { return NULL; }               // fb не е затворен преди return!
[OK]  if (!f) { fclose(fb); return NULL; }   // затвори файла преди return
```